第十六讲教学设计：工程化与部署实战 (20分钟)

课程信息

- **课程名称**: Java Web应用开发 (Java Web Application Development)
- **教学方式**: 双语教学（英文授课，关键术语中英文对照）
- **教学主题**: 工程化与部署实战——统一异常与日志、全局错误页、JNDI数据源与配置分离、Tomcat打包部署
- **授课时长**: 20分钟
- **适用对象**: 已掌握Servlet/JSP、Filter/Listener、DAO/事务与连接池基础的学生

一、教学目标

知识目标

1. 理解"配置分离（Configuration Separation）、环境解耦（Environment Decoupling）、可观察性（Observability: 日志Logging/错误页Error Page）"的工程化要求。
2. 掌握统一异常（Unified Exception Handling）与统一日志（Unified Logging）的落地方式（过滤器Filter/拦截器Interceptor思路）。
3. 掌握 `web.xml` 全局错误页（Global Error Page）配置；理解与统一异常处理的配合关系。
4. 掌握在Tomcat中配置JNDI数据源（JNDI DataSource），并在Web应用中通过JNDI查找 `DataSource` 。
5. 掌握打包WAR（WAR Package）与部署（Deployment）到Tomcat的基本流程。

能力目标

1. 能对现有Web应用接入统一日志与统一异常处理，输出规范日志并友好兜底。
2. 能将数据库连接信息从代码中剥离到容器的JNDI资源中，实现环境配置分离。
3. 能完成项目的WAR打包、部署、验证与回滚。

素质目标

1. 强化工程规范与上线意识，形成“可运行、可观测、可运维”的完整闭环。
2. 提升问题定位与应急处置能力（错误页兜底、日志追踪）。
3. **[思政融合]** 强调规则与秩序：日志留痕、责任可追，保障公共系统的公平与安全，体现现代治理能力与法治精神。

二、教学重点与难点

教学重点

- 统一异常与全局错误页的职责边界；日志格式与关键信息（时间、级别、TraceID、URI、用户）。
- JNDI数据源配置（ `context.xml` / `server.xml` ）与应用侧 `web.xml` 的 `resource-ref` 。
- WAR打包与部署验证流程。

教学难点

- 过滤器统一捕获异常与容器错误页的优先级与配合。
- 多环境（dev/test/prod）配置分离与变更管理。
- 部署后的排错与回滚策略。

三、教学内容与时间安排

第一部分：问题引入——从课堂Demo到可上线应用 (3分钟)

内容要点

1. 展示“硬编码数据库配置、控制台零散日志、异常白屏”的痛点。
2. 目标：配置外置、异常兜底、日志规范、可一键部署。

教学方法

- 反例对比 + 目标导向。

第二部分：统一异常与统一日志、全局错误页 (9分钟)

内容要点

1. 统一异常过滤器（核心思路）：

```
@WebFilter("/*")
public class GlobalExceptionFilter implements Filter {
    public void doFilter(ServletRequest r, ServletResponse s, FilterChain c)
            throws IOException, ServletException {
        HttpServletRequest req = (HttpServletRequest) r;
        long startedAt = System.currentTimeMillis();
        String uri = req.getRequestURI();
        try {
            c.doFilter(r, s);
        } catch (Throwable ex) {
            // 1) 记录错误日志（最少包含时间/URI/用户/异常栈）
            ex.printStackTrace(); // 生产中请接入标准日志框架
            // 2) 兜底到统一错误页
            ((HttpServletResponse) s).sendRedirect(req.getContextPath() +
"/jsp/500.jsp");
        } finally {
            long cost = System.currentTimeMillis() - startedAt;
            System.out.println("[ACCESS] uri=" + uri + " cost=" + cost + "ms");
        }
    }
}
```

2. 全局错误页（`web.xml`）：

```
<error-page>
    <error-code>404</error-code>
    <location>/jsp/404.jsp</location>
</error-page>
<error-page>
    <error-code>500</error-code>
```

```
    <location>/jsp/500.jsp</location>
</error-page>
```

- 过滤器兜底与错误页不冲突：先由过滤器记录日志与重定向；或由容器转到错误页。

3. 错误页示例（`500.jsp`）：

```
<h2>抱歉，系统繁忙，请稍后再试</h2>
<p>时间：${pageContext.request.time}</p>
<p>如持续出现，请联系管理员并提供访问时间与页面地址。</p>
```

教学方法

- 代码走查 + 故障演练（制造异常，观察日志与错误页）。

第三部分：配置分离与JNDI数据源、WAR打包部署 (6分钟)

内容要点

1. 在Tomcat配置JNDI数据源（`conf/context.xml` 或应用 `META-INF/context.xml`）：

```
<Resource name="jdbc/demoDS" auth="Container" type="javax.sql.DataSource"
          maxTotal="50" maxIdle="5" maxWaitMillis="10000"
          username="root" password="123456"
          driverClassName="com.mysql.cj.jdbc.Driver"
          url="jdbc:mysql://localhost:3306/demo?useSSL=false&serverTimezone=UTC"/>
```

2. 应用侧声明引用（`web.xml`）：

```
<resource-ref>
    <description>Demo DataSource</description>
    <res-ref-name>jdbc/demoDS</res-ref-name>
    <res-type>javax.sql.DataSource</res-type>
    <res-auth>Container</res-auth>
</resource-ref>
```

3. 代码查找与使用：

```
Context init = new InitialContext();
DataSource ds = (DataSource) ((Context)
init.lookup("java:comp/env")).lookup("jdbc/demoDS");
try (Connection conn = ds.getConnection()) {
    // ... 数据库操作
}
```

4. WAR打包与部署：
   - IDE/构建工具打包WAR（`myapp.war`）
   - 复制到Tomcat的 `webapps/` 目录

- 启动 `bin/startup.bat` 或 `startup.sh` 并访问验证

教学方法

- 步骤演示 + 现场联调。

第四部分：课堂总结与展望 (2分钟)

总结要点

1. 统一异常 + 错误页：稳定、可观测、可告警。
2. JNDI数据源：配置与代码解耦，便于多环境切换。
3. WAR部署：形成可交付产物与标准流程。

展望（承上启下）

- 建议课后将项目整理为“可部署”的最小产品（MVP），并撰写上线手册与回滚预案。

四、教学方法与手段

教学方法

1. **问题驱动 (Problem-Driven)**: 从Demo到生产应用的差距引入
2. **代码走查 (Code Walkthrough)**: 讲解异常处理和日志实现
3. **演示演练 (Demo & Practice)**: JNDI配置和WAR部署演示
4. **对比分析 (Comparison)**: 硬编码 vs 配置分离对比
5. **双语教学法 (Bilingual Teaching)**:
    - 课堂采用英文授课，关键技术术语提供中英文对照
    - 重要概念（Configuration Separation, Observability, Exception Handling, JNDI, Deployment）首次出现时给出英文表达和中文释义
    - 鼓励学生使用英文术语描述工程化实践
    - 代码注释和变量命名使用规范英文

教学手段

- IDE (IntelliJ IDEA/Eclipse)
- Tomcat服务器 (Tomcat Server)
- 浏览器开发者工具 (Browser DevTools)
- 日志控制台 (Log Console)
- PPT/Marp课件 (Presentation Slides)

五、教学评价

- **过程性评价**: 是否能说明统一异常与错误页的差异与协作；是否能正确画出JNDI接入流程图。
- **终结性评价**: 是否能将现有项目改造为JNDI数据源接入、具备统一日志与错误页，并成功打包部署运行。

六、课后拓展

思考题

1. 当异常发生在重定向之后会怎样？如何保证日志完整与错误页友好？

2. 多环境（dev/test/prod）如何管理 `context.xml` 的差异？能否通过外部化配置与环境变量统一管理？

实践作业

1. 为应用加入“请求TraceID”（可用UUID）并贯穿日志打印与错误页提示，提升问题追踪效率。
2. 将直连 `DriverManager` 的代码彻底替换为JNDI数据源，并提交部署指南（含回滚步骤）。